

OUTLINE

- Abstract
- Introduction
- Problem Statement
- Existing System
- Disadvantages
- Proposed System
- Advantages
- System Requirements
- System Architecture
- Modules
- UML Diagrams
- Output Screenshots
- Testing Screenshots
- Future Enhancements

ABSTRACT

- Intelligent underwater object detection system that tackles poor visibility and color distortion using image denoising, dehazing, and contrast correction. It uses YOLO for fast and accurate detection, while Grad-CAM highlights which parts of the image influenced the model's decision. By preprocessing images with GMAN dehazing, the system enhances detection accuracy under challenging underwater conditions, such as varying lighting and visibility levels. Results, including dehazed images and detected objects, are displayed in a structured manner. To make the system more user-friendly, we added a chatbot that lets users ask questions about detected objects in natural language and get clear, helpful answers. This combination of detection, explanation, and conversation makes underwater exploration more transparent, interactive, and accessible for researchers and developers.

INTRODUCTION

- Underwater environments are crucial for marine research, defense, and ecological monitoring.
- This overcomes visibility issues like contrast, color distortions which reduces identification.
- A robust underwater object detection system enables real-time identification of submerged objects, even in degraded conditions

PROBLEM STATEMENT

- Underwater object detection faces significant challenges due to poor visibility, light scattering, and color distortion caused by water conditions. These factors degrade image quality and reduce the accuracy of traditional computer vision models. The dynamic nature of underwater environments such as moving particles, varying lighting, and unpredictable object shapes further complicates detection.
- These effects obscure fine details and boundaries, causing standard object detection models trained primarily on clear land-based data—to perform poorly, with frequent missed targets or incorrect classifications. Current approaches often apply detection directly on raw degraded frames, leading to unreliable outcomes in real applications like environmental surveys or infrastructure maintenance.
- As a result, reliable identification in underwater scenes remains a complex and unsolved problem for applications like exploration, navigation, and environmental monitoring.

EXISTING SYSTEM



- Existing system depends on manual observation, edge detection and classical image processing which are less accurate in underwater conditions.
- Previous models struggles to detect multi scale objects in the image.
- Inconsistent across varying underwater environments.
- Reduced accuracy in low-visibility or turbid water.
- Due to inherent challenges in aquatic imaging such as light scattering, absorption, color distortion, and reduced contrast performance remains inconsistent.

DISADVANTAGES



- High manual effort and time consumption.
- Low accuracy than modern versions.
- Lack off explainability.
- Poor generalization across diverse underwater conditions.

PROPOSED SYSTEM

- The proposed system presents an advanced, automated pipeline that first enhances degraded underwater images through dehazing using the Generic Model-Agnostic Convolutional Neural Network (GMAN), effectively reducing haze-like effects and restoring clarity, contrast, and natural colors.
- The enhanced output is then processed by the state-of-the-art YOLOv11 model for fast and precise multi-object detection in both static images and live video streams. The integrated approach ensures superior robustness across varied aquatic environments.
- Included Grad-CAM to generate heatmaps that highlight the most important regions of an image.

ADVANTAGES



- Real-time object detection.
- Handles poor lighting and noise.
- Faster and more accurate results.
- Chatbot enables intuitive, human friendly interactions.
- Enhanced image quality.
- Robust multi-object detection.

SYSTEM REQUIREMENTS

Hardware Requirements :

Processor : Intel i5 or higher

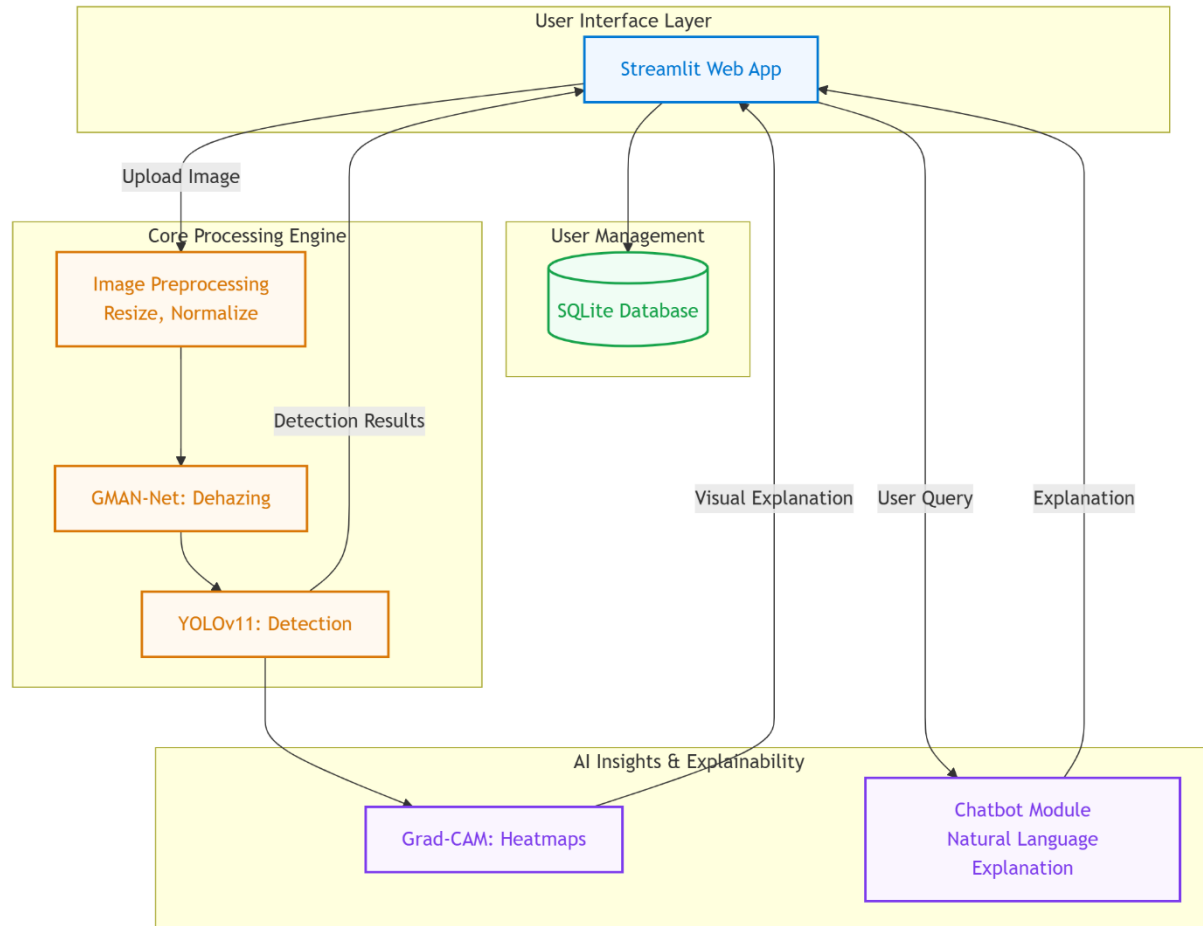
RAM : 8GB or more

Hard disk : 160GB or more

SOFTWARE REQUIREMENTS

Operating System	: Windows 10/11 / Linux/ macOS
Programming language	: Python
Libraries	: Numpy, Pandas, Tensorflow, Keras, Torch, Ultralytics, OpenCV
Front end	: Streamlit library
IDE	: google Colab/ VS Code
Environment Manager	: MiniConda

System Architecture



Modules



- User Module
- System Module
- Interpretation & chatbot layer

User Module

- This module manages the human computer interface, ensuring security and a personalized experience.
- This module handles secure Login and Registration. It manages “valid” vs “invalid” credentials to secure dataset and results.
- User can upload raw underwater images for processing.
- A visual interface where the user views the output (e.g., images with bounding boxes around detected fish, shipwrecks, or debris).

System Module

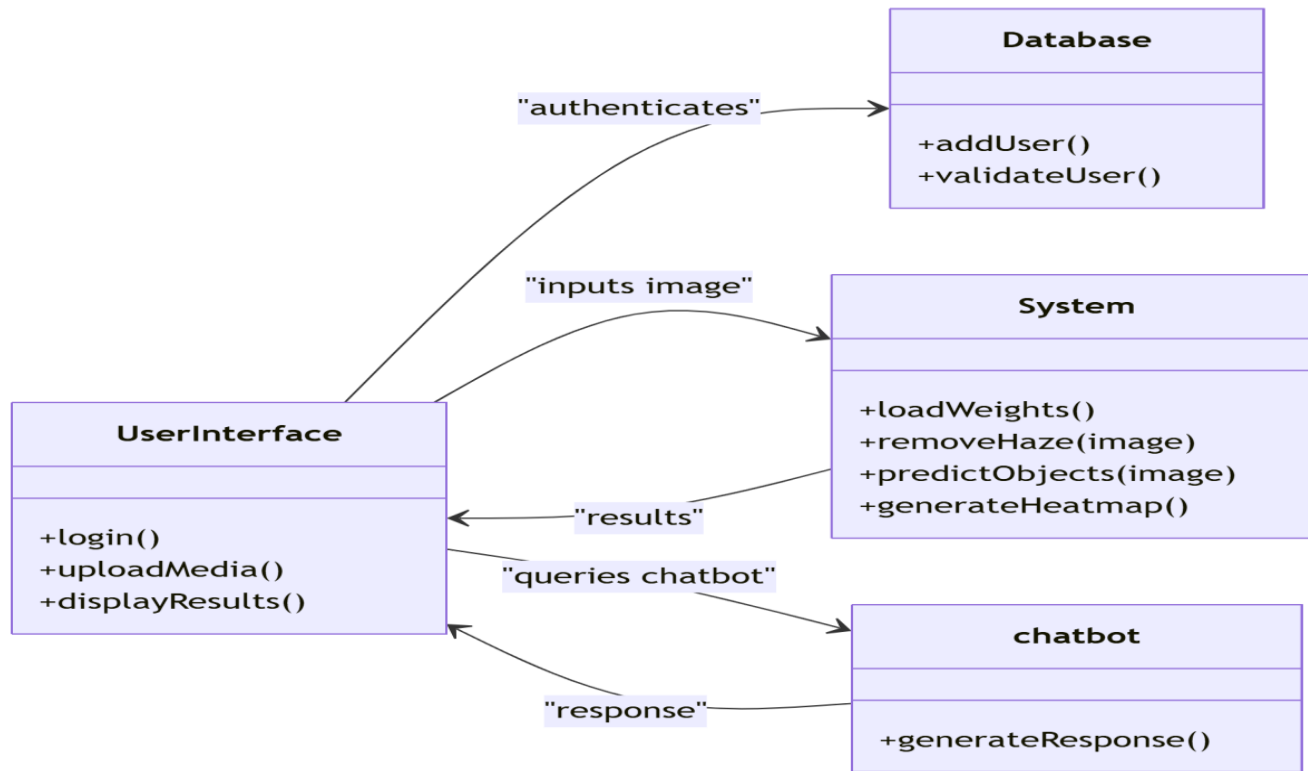
- **System:** The core component where deep learning models (e.g., YOLO) process data and perform object detection.
- **Algorithm Training:** The core phase where the model learns to differentiate between objects
- **Real-Time Analytics:** Utilizes a single-stage regression architecture to predict bounding boxes and class probabilities in one pass.
- **The Restoration Engine (GMAN Dehazing):**
 1. Uses a fully convolutional network that avoids reliance on traditional physical scattering models.
 2. Removes haze and restores natural colors lost due to underwater absorption and scattering.
 3. Employs an encoder-decoder with residual blocks to sharpen boundaries before passing data to the detection engine.

Chatbot Module

- **Chatbot:** "The Conversational Assistant" This module acts as the "Middleman," translating complex technical data into simple human conversation.
- **Grad-CAM Interpretability:** Visualizes the model's decision-making by generating heatmaps that show exactly where the YOLOv11 model focused its attention.
- Provides additional information about detected objects, such as environmental facts about a specific species of fish detected

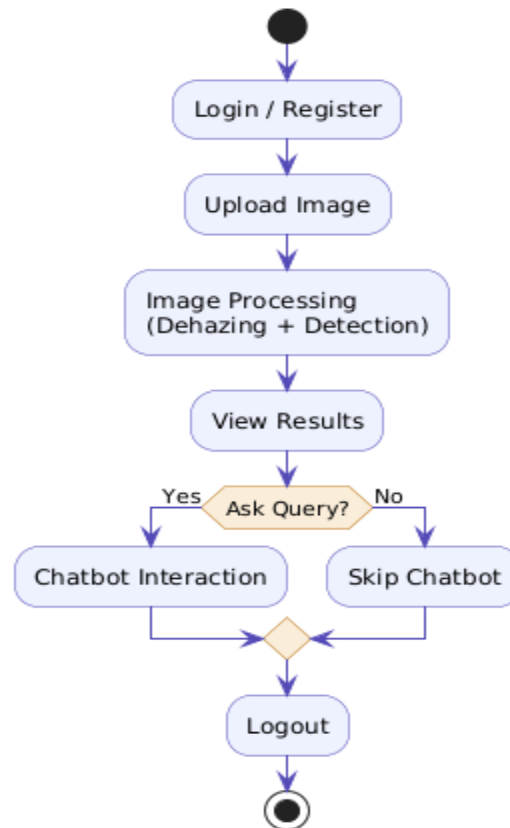
UML Diagrams

- Class Diagram :



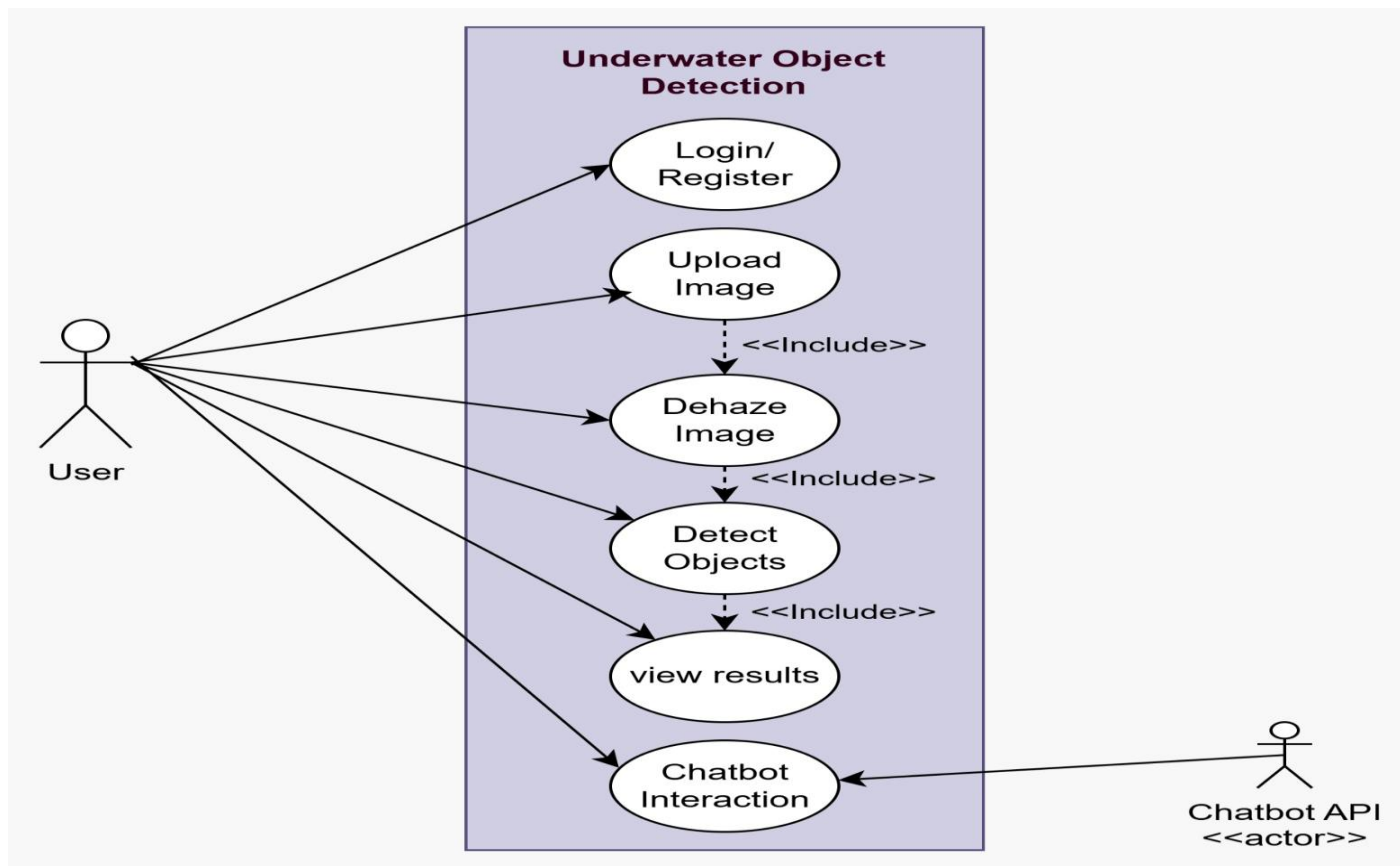
UML Diagrams

- Activity Diagram:



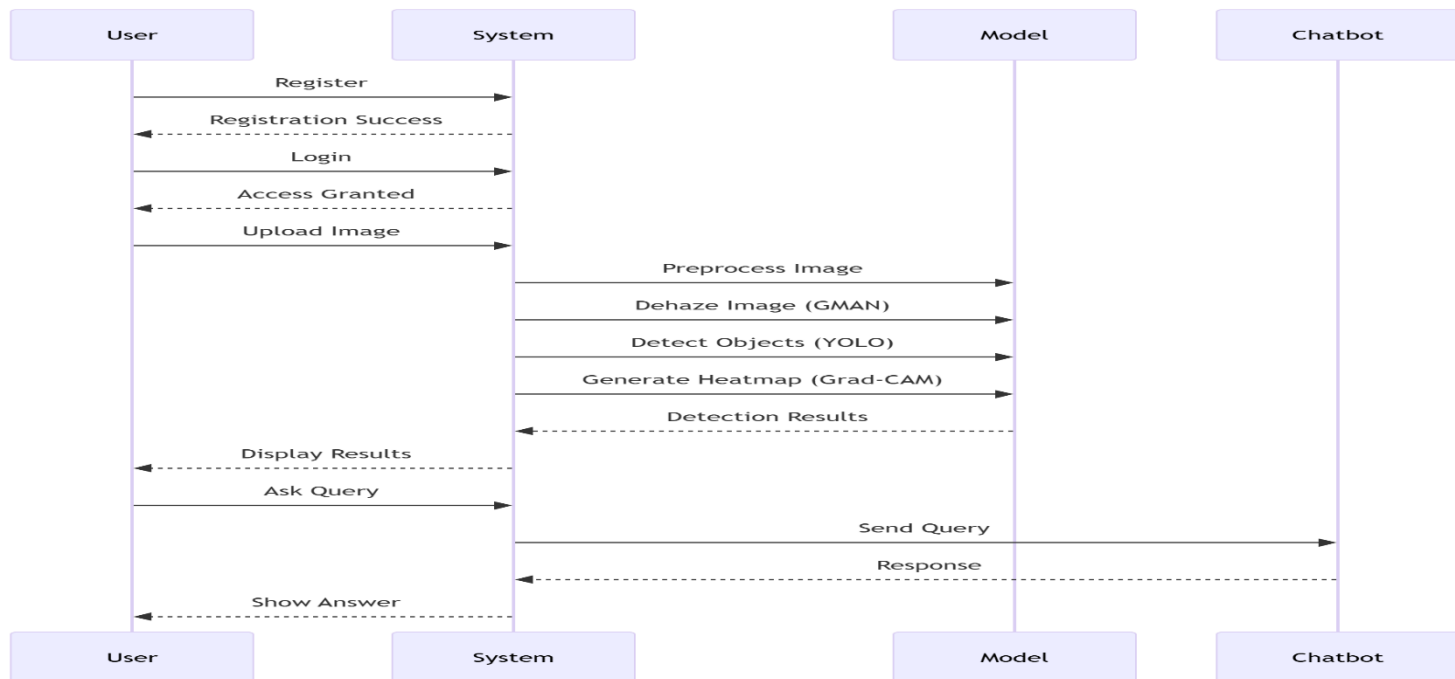
UML Diagrams

- Use case diagram:

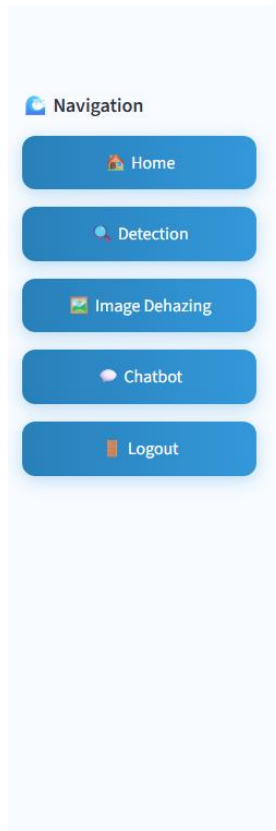


UML Diagrams

- Sequence diagram:



Output Screenshots



Deploy ⋮



✓ Found 2 object(s)!

Detection Details

1. animal_crab — Confidence: 0.85
2. rov — Confidence: 0.81

Navigation

Home

Detection

Image Dehazing

Chatbot

Logout

Deploy

vid_000035_frame0000071_jpg.rf.25547d9b5abf1c2ba9f3e863369c355e.jpg 52.5KB



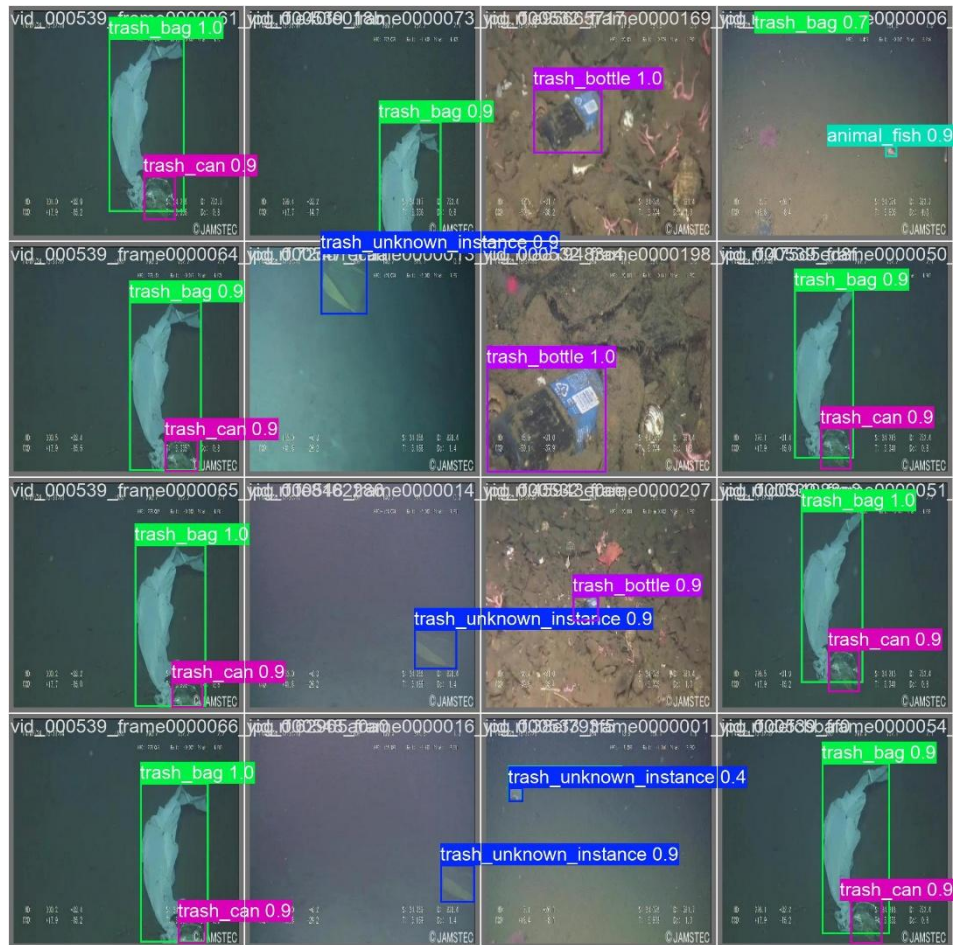
Original Hazy Image



Dehazed Enhanced Image

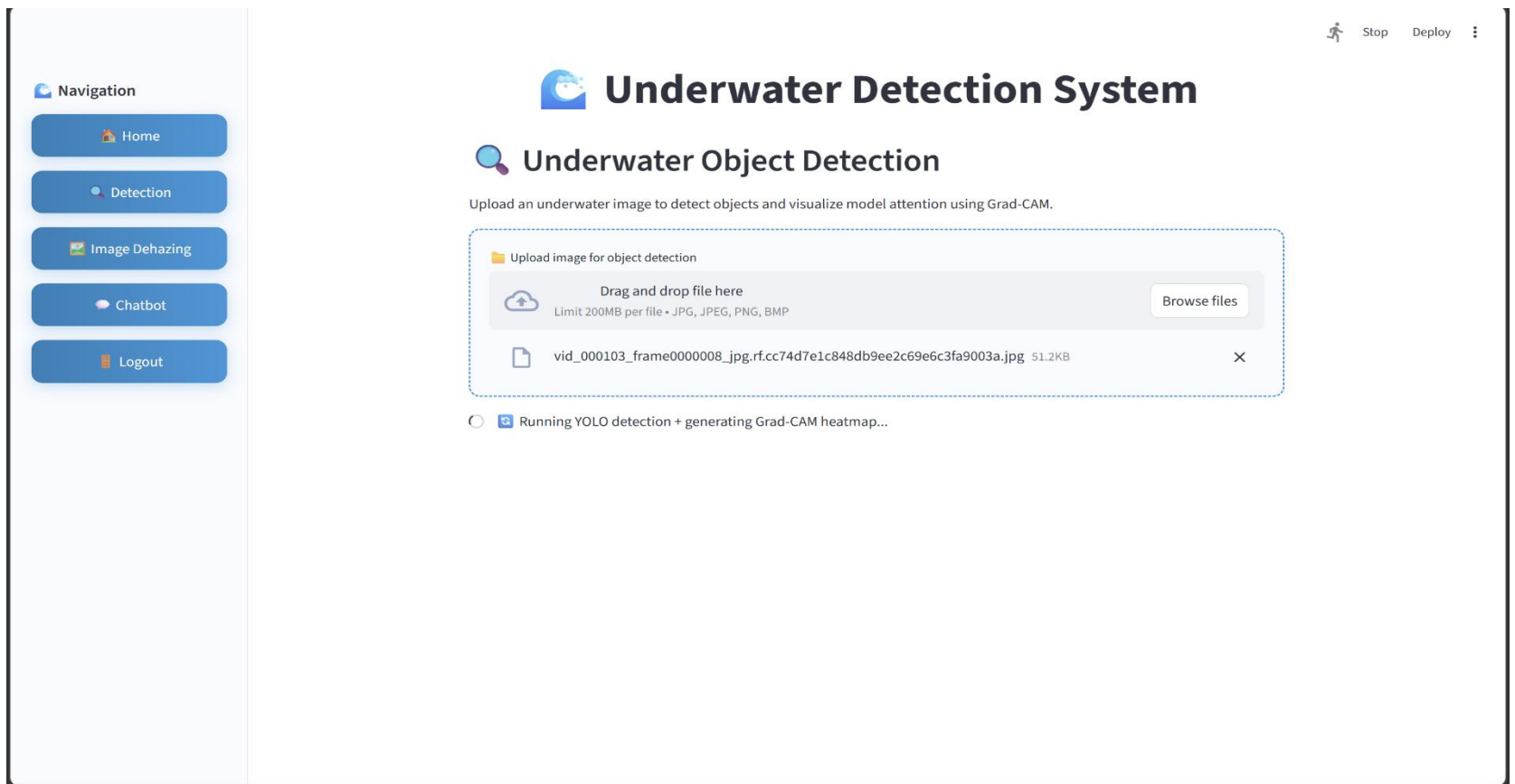
Image successfully dehazed!

Download Dehazed Image



Testing Screenshots

Page to upload image for object detection



Testing Screenshots

Output screen

The screenshot displays a Streamlit web application running on a browser at localhost:8501. The interface includes a left-hand navigation menu with buttons for Home, Detection, Image Dehazing, Chatbot, and Logout. The main content area shows the results of an object detection task. At the top, a file upload bar indicates a limit of 200MB per file and shows the uploaded file: vid_000103_frame0000008_jpg.rf.cc74d7e1c848db9ee2c69e6c3fa9003a.jpg (51.2KB). Below the upload bar, two side-by-side images are displayed. The left image is the original photo, and the right image is a Grad-CAM heatmap showing model attention. Both images show a robot on the left and an eel on the right. The robot is labeled 'rov' with a confidence of 0.92, and the eel is labeled 'animal_eel' with a confidence of 0.86. A green bounding box is drawn around the eel in both images. Below the images, a green status bar indicates 'Found 2 object(s)!'. At the bottom, a section titled 'Detection Details' lists the detected objects: 1. rov — Confidence: 0.92 and 2. animal_eel — Confidence: 0.86.

Limit 200MB per file • JPG, JPEG, PNG, BMP

vid_000103_frame0000008_jpg.rf.cc74d7e1c848db9ee2c69e6c3fa9003a.jpg 51.2KB

rov 0.92

animal_eel 0.86

rov 0.92

animal_eel 0.86

Detected Objects (with bounding boxes)

Grad-CAM Heatmap (Model Attention)

Found 2 object(s)!

Detection Details

1. rov — Confidence: 0.92
2. animal_eel — Confidence: 0.86

Testing Screenshots

- Page to upload image for dehazing

Navigation

- Home
- Detection
- Image Dehazing
- Chatbot
- Logout

Deploy

Underwater Image Dehazing

Upload a hazy underwater image to enhance clarity and remove haze using deep learning.

Upload image for dehazing

Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG, BMP

Browse files

vid_000319_frame0000029_jpg.rf.31812e68ac75ca73570f0fb2128bcb40.jpg 31.8KB

Original Hazy Image

Dehazed Enhanced Image

Image successfully dehazed!

Testing Screenshots

Chat-bot interpretation about model prediction



Based on these results, here's an explanation of what your model is predicting:

Your model is performing a **binary classification task**, meaning it's trying to distinguish between two categories: `rov` (presumably a Remotely Operated Vehicle) and `animal_starfish`.

Here's a breakdown of its predictions:

1. Dominant Prediction: `animal_starfish`

- Out of 17 predictions, **13 are** `animal_starfish` and only **4 are** `rov`.
- This suggests a strong tendency for your model to classify items as `animal_starfish`. This could be because:
 - The input data predominantly contains starfish.
 - The model has a bias towards the `animal_starfish` class.
 - The `rov` class is harder for it to identify.

2. Confidence Levels - High:

- The model shows **high confidence** in some of its predictions for *both* classes:
 - `rov` with 0.83 and 0.67 confidence.
 - `animal_starfish` with 0.77, 0.74, and 0.61 confidence.
- This indicates that for certain inputs, the model is quite sure about its classification.

Future Enhancements

- While the current system provides a robust framework, the following enhancements would further elevate its utility:

A. Real-Time Edge Deployment

- **Optimization:** Convert the YOLOv11 model to **TensorRT** or **ONNX** formats to enable real-time inference on edge devices like autonomous underwater vehicles (AUVs).
- **Temporal Tracking:** Integrate algorithms like DeepSORT to track moving marine life across video frames, preventing duplicate counts.

B. Voice Commands: Implement a voice interface for hands-free interaction during field research.

C. Active Marine Robotics Integration: Transition the system from a passive monitoring tool to an active exploration platform by deploying the YOLOv11 model onto **Autonomous Underwater Vehicles (AUVs)**